

Removable and Irreducible: A Token-Cost Ledger for the Multilingual Tokenization Tax

Anonymous ACL submission

Abstract

Large language models pay a well-documented tax on non-English text: the same content costs several times more tokens, and because attention is quadratic in sequence length, far more compute. We ask how much of this tax is removable. Framing the token layer as source coding—transformer compute is monotone in sequence length, whose per-atom floor is the Shannon rate $H/\log_2 V$, an object already applied to tokenizers in prior work—we assemble a token-cost ledger that splits each language’s cost, at fixed parallel content, into a removable coding redundancy, a residual coding slack, an intrinsic-content term, and an orthogonal, irreducible grapheme-to-phoneme term that governs the multimodal rather than the text cost. On FLORES-200 across eight languages, a production tokenizer costs up to $8.9\times$ more tokens for Indic scripts than for English; a script-matched code trained on 1,012 sentences removes a median 64% of that excess (bootstrap 95% CI [0.638, 0.647]), and a script-fair information floor shows the intrinsic content differs by under 6%—the tax is representational, not informational. A constructed code removes 98% of a controlled source’s redundancy, and the token tax implies up to $79\times$ attention cost. We are explicit about scope and failure: this is compute-and-memory accounting, not a model-quality claim; we neither measure nor claim the cross-lingual direction of the orthographic term; and our matched code is a conservative small-data demonstration. We contribute the unifying ledger, the removable-versus-intrinsic attribution, and an open one-command harness.

1 Introduction

It costs more to say the same thing to a language model in Telugu than in English. On identical, professionally-translated content, a widely-used production tokenizer emits up to $8.9\times$ as many tokens for an Indic script as for English (Section 4); an older English-centric tokenizer reaches $16\text{--}20\times$

per word. Because self-attention is $\Theta(N^2)$ in sequence length N (Vaswani et al., 2017), a $k\times$ token inflation is a $k^2\times$ attention-compute inflation and a $k\times$ shrink of the effective context window. This multilingual “token tax” is well documented as a phenomenon and a fairness problem (Ahia et al., 2023; Petrov et al., 2023; Lundin et al., 2025).

The question this paper asks is narrower and, we think, more useful: *how much of the tax is removable by choosing a better code, and how much is intrinsic to the language?* We answer it by assembling a *token-cost ledger*. Holding semantic content fixed with a parallel corpus, we decompose each language’s normalized sequence length into (i) a *removable* coding redundancy—the penalty for using a code fit to the wrong (English-dominated) distribution; (ii) a residual coding slack from a finite, finitely-trained vocabulary; (iii) an *intrinsic*-content term measured by a script-fair compressor; and, on an orthogonal axis, (iv) an irreducible grapheme-to-phoneme term that does not affect text sequence length at all but governs the multimodal (speech) ledger. Terms (i)–(iii) live on the text-compute axis; (iv) is the organic cost of a deep orthography.

None of the ledger’s individual pieces is new. The fertility floor $F^* = H/\log_2 V$ is Shannon source coding (Shannon, 1948; Cover and Thomas, 2006), applied to tokenizers as an “efficiency” or “capacity-utilization” metric by Zouhar et al. (2023) and Erdogan et al. (2026), the latter of whom also give the additive floor-plus-redundancy split we use. Orthographic depth as an information-theoretic quantity is established (Torres and Futrell, 2025). A compute-optimal vocabulary size is derived by Tao et al. (2024). Our contribution is (a) to *unify* the removable text term and the irreducible orthographic term into one cost ledger; (b) to *attribute* the observed multilingual tax empirically to removable versus intrinsic components on real parallel data, with a constructed code that drives the remov-

able term to zero; and (c) to keep the whole thing scoped strictly to *compute and memory*, which is what makes the accounting well-posed.

Scope fence (stated once, up front). This is a cost accounting, not a quality claim. We do *not* assert that fewer tokens make a better model; Schmidt et al. (2024) and others show it need not, and we agree. Everything here concerns FLOPs, KV-cache bytes, and context-window occupancy, quantities that are monotone in sequence length regardless of downstream accuracy.

Contributions.

- A **unified token-cost ledger** (Section 2) putting the removable coding redundancy and the irreducible orthographic term on one accounting identity, with a single estimand—the removable fraction ρ .
- An **empirical attribution** (Section 4) on FLORES-200: a script-matched code trained on 1,012 sentences removes a median $\rho = 0.64$ of the production tax (CI [0.638, 0.647]), and the script-fair floor shows intrinsic content varies by $< 6\%$ across Indic languages.
- A **constructed floor-approaching code** and the quadratic-cost consequence (Section 5): it removes 98% of a controlled source’s redundancy, sitting 0.036 bits above the entropy floor; the tax implies up to $79\times$ attention cost.
- An **open, one-command harness** and an honest limitations section (Section 7), including a pre-registered control we report as *not* established.

2 The token-cost ledger

Cost is monotone in sequence length. A decoder-only transformer of width d and L layers processing N tokens pays $\Theta(LN^2d)$ attention FLOPs, $\Theta(LNd^2)$ feed-forward FLOPs, $\Theta(LNd)$ KV-cache memory, and an $O(Vd)$ vocabulary term (Vaswani et al., 2017).

Proposition 1 (cost monotonicity). *For fixed (d, L, V) , transformer compute and KV memory for a fixed piece of content are non-decreasing in N , and strictly increasing once $N \gtrsim d$ (the attention term dominates).*

Every term is non-decreasing in N ; the N^2 term’s derivative $2LNd$ overtakes the linear terms once $N > O(d)$. The consequence is the objective: *minimize expected N for fixed content*. In the quadratic

regime a sequence-length ratio r is an attention-cost ratio r^2 .

The floor (prior art). Model content as atoms drawn from a source p with entropy $H(p)$ bits/atom, encoded to tokens over a V -ary alphabet.

Lemma 1 (fertility floor). *Any uniquely-decodable V -ary code has expected tokens per atom $F \geq H(p)/\log_2 V =: F^*$, with equality approached within one token by a matched Huffman code and exactly in the block limit. If the code is optimal for a wrong distribution q , then $F = F^* + R$ with $R = D_{\text{KL}}(p \parallel q)/\log_2 V \geq 0$.*

We state Lemma 1 for self-containedness and attribute it (Cover and Thomas, 2006; Zouhar et al., 2023; Erdogan et al., 2026); $\eta = H/\log_2 V$ is Erdogan et al.’s capacity utilization, and the $F^* + R$ split is their Appendix-C decomposition. R is the *removable* part: refit the code to p and $R \rightarrow 0$.

The ledger (our synthesis). Hold content fixed with a parallel corpus and normalize every quantity to a reference language (English = 1). Write $\text{NSL}_E(L)$ for the total tokens of language L under encoder E divided by English’s under the same encoder. Then the excess of a production encoder decomposes additively into non-negative, individually measurable terms:

$$\begin{aligned} \text{NSL}_{\text{prod}}(L) - 1 &= [\text{NSL}_{\text{prod}} - \text{NSL}_{\text{match}}] \\ &\quad + [\text{NSL}_{\text{match}} - \text{NSL}_{\text{floor}}] \quad (1) \\ &\quad + [\text{NSL}_{\text{floor}} - 1], \end{aligned}$$

the three terms being removable redundancy (vocabulary mismatch), residual coding slack, and intrinsic content; and, on an orthogonal axis, an irreducible term $G(L) = H(\text{phoneme} \mid \text{grapheme})$ that leaves text N untouched but governs the speech ledger. The unification of the removable text term with the irreducible orthographic term is the contribution; neither half is ours (the removable term is Erdogan et al., 2026; orthographic depth as algorithmic mutual compressibility is Torres and Futrell, 2025, whose Kolmogorov instrument we replace with a Shannon one in the LLM-cost setting). The headline estimand is the *removable fraction*

$$\rho(L) = 1 - \frac{\text{NSL}_{\text{match}}(L) - 1}{\text{NSL}_{\text{prod}}(L) - 1}, \quad (2)$$

the share of the production tax a language-matched code removes: $\rho \rightarrow 1$ means the tax is a code artifact (synthetic), $\rho \rightarrow 0$ means it is intrinsic (organic). Substituting $N = A H/\log_2 V$ into the

cost model yields an interior compute-optimal vocabulary V^* , but this is already established empirically (Tao et al., 2024; Limisiewicz et al., 2026); we cite it and claim nothing here.

3 Experimental setup

Real parallel data. FLORES-200 (NLLB Team et al., 2022) (CC-BY-SA), professionally translated and sentence-aligned, so line i of every language file is the same sentence. We report on eight languages: a Latin control (English, Spanish, German), Devanagari (Hindi), Bengali, and three Dravidian abugidas (Telugu, Tamil, Kannada), measuring on the 1,012-sentence `devtest` split. **Encoders.** UTF-8 bytes; Unicode extended grapheme clusters (the akshara-preserving atom); three production byte-level BPE tokenizers (GPT-2, `cl100k_base`, `o200k_base`); and a per-language *matched* BPE trained *held out* on the FLORES dev split and evaluated on `devtest`. **Information floor.** A script-fair content estimate: LZMA over the grapheme-cluster-ID stream (not UTF-8 bytes), so a script is not charged for its 3-byte-per-codepoint UTF-8 assignment. **Constructed source.** A Zipf($s=1.1$) source over 256 concepts with a known entropy. **Apparatus.** Single-thread Python; `tiktoken` and the standalone `tokenizers` library (no GPU, no model weights); deterministic seeds; one command regenerates every number and figure.

Pre-registration. Hypotheses, decision rules, and four negative controls were frozen before the confirmatory run. The calibration control (NC2) requires the stack to recover a *known* entropy: on a uniform-32 source it returns $\hat{H} = 5.0000$ bits and a Huffman length in $[H, H+1]$; it passed before any language number was read.

4 Results: the removable tax

Table 1 and Figure 1 report the ledger. Under `cl100k_base` the production tax reaches $8.29\times$ (Telugu), $8.86\times$ (Kannada), and $4.76\times$ (Hindi); under the older GPT-2 tokenizer, fertility per word reaches $16.4\text{--}20.3\times$ for Dravidian scripts. A *script-matched* code trained on only 1,012 sentences (learned vocabulary 2,000–3,000) brings these to $2.89, 2.95,$ and $2.54\times$; and the script-fair information floor sits at $1.02\text{--}1.06\times$ —the intrinsic content of the same sentences is within 6% across all Indic languages. Bootstrapping sentences, the median removable fraction across the five Indic languages

Language	b/char	NSL _{prod}	NSL _{mat}	ρ
English	1.00	1.00	1.00	—
Spanish	1.02	1.54	1.15	0.72
German	1.02	1.59	1.13	0.78
Hindi	3.92	4.76	2.54	0.59
Bengali	4.22	5.80	2.72	0.64
Telugu	4.62	8.29	2.89	0.74
Tamil	4.23	7.64	3.39	0.64
Kannada	4.15	8.86	2.95	0.75
Indic median				0.64

Table 1: Token-cost ledger on FLORES-200 `devtest`, production tokenizer `cl100k_base`. NSL = normalized sequence length (English = 1); the script-fair floor NSL_{floor} is $1.02\text{--}1.06$ for Indic. ρ (Eq. 2) uses a matched code trained held-out on $\sim 1\text{k}$ sentences (a conservative demonstration; Indic median CI $[0.638, 0.647]$).

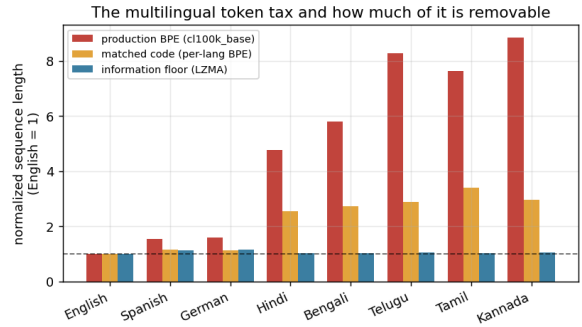


Figure 1: The multilingual token tax and how much of it is removable. Production BPE vs. a script-matched code vs. the script-fair information floor, normalized to English.

is $\rho = 0.642$ with a 95% CI of $[0.638, 0.647]$: a matched code removes about two-thirds of the production tax, and the information floor shows nearly all of the rest is coding slack rather than content. That production tokenizers themselves disagree by $4\times$ on the same Telugu content ($8.29\times$ under `cl100k` vs. $1.93\times$ under the more multilingual `o200k`) is independent evidence that the tax is a property of the code, not the language.

Bytes-per-char predicts the tax (a replication).

Across the eight languages, UTF-8 bytes-per-character correlates with the production tax at Spearman $\rho_s = 0.83$ ($p = 0.01$), replicating Ahia et al. (2023) and Petrov et al. (2023); our addition is the decomposition, not the correlation.

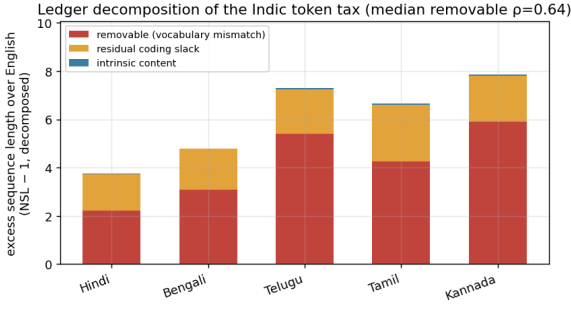


Figure 2: Ledger decomposition of the Indic token tax (Eq. 1): removable coding redundancy dominates; residual coding slack is removable in principle; intrinsic content is a sliver.

5 Decomposition, a constructed code, and quadratic cost

Figure 2 decomposes the Indic excess of Eq. 1. The removable term dominates; the intrinsic-content sliver is ≤ 0.06 in every case. The residual coding slack—the gap between our small matched code and the floor—is itself removable in principle, so it is a lower bound on removability, not a second intrinsic term; that a 200k-vocabulary production tokenizer ($\circ 200k$) already reaches $1.93\times$ on Telugu confirms the slack is training, not content.

A constructed code hits the floor. On the controlled Zipf source (entropy $H = 5.77$ bits/concept), a mismatched fixed-width code pays 8.0 bits/concept; the matched Huffman code pays 5.80—0.036 bits above the floor, within Lemma 1’s one-bit guarantee—removing 98% of the redundancy (Figure 3, left). We deliberately do *not* report an empirical block code beating the floor: at block sizes > 1 the k -gram alphabet is undersampled and an in-sample Huffman would appear to beat H from finite-sample bias. We position this construction as the discrete-code limit of byte-entropy patching (Pagnoni et al., 2024), not a proposed human language.

Quadratic amplification. Because attention is $\Theta(N^2)$, the token tax is amplified in compute: the same content costs up to $79\times$ (Kannada) and $69\times$ (Telugu) the attention work of English under $\circ 1100k$, collapsing to $8\text{--}11\times$ under the matched code (Figure 3, right).

6 Related work

Information theory of tokenizers. Zouhar et al. (2023) frame the tokenizer as a channel and define

an entropy-to-max-entropy efficiency (the parent of $H/\log_2 V$); Erdogan et al. (2026) define capacity utilization $\eta = H_1/\log_2 K$ and the additive floor-plus-redundancy decomposition we build on; Rajaraman et al. (2024) and Gastaldi et al. (2025) give complementary theories. We contribute neither the floor nor the split—we contribute their *use* as a multilingual accounting. **The multilingual tax.** Ahia et al. (2023), Petrov et al. (2023), and Lundin et al. (2025) document the cross-language disparity; Indic-specific tokenizers (IndicSuperTokenizer authors, 2025) reduce fertility. We explain the disparity as a removable KL redundancy above a script-fair floor, and note that Petrov et al.’s residual byte-level disparity is exactly the intrinsic + orthographic remainder our ledger predicts. **Byte-level models.** MEGABYTE (Yu et al., 2023) and the Byte Latent Transformer (Pagnoni et al., 2024) remove the discrete code; our ledger *explains* what they can remove (R) and cannot (intrinsic, G). **Compression is not quality.** Schmidt et al. (2024) and Bostrom and Durrett (2020) show fewer tokens need not improve models; our scope fence makes this orthogonal. **Orthographic depth.** Torres and Futrell (2025) formalize transparency as Kolmogorov mutual compressibility; we borrow the quantity (as Shannon $H(\text{phoneme} \mid \text{grapheme})$) for the irreducible axis.

7 Limitations and honest negatives

(1) **We did not establish the orthographic direction.** We pre-registered a control (NC4) that English’s grapheme-to-phoneme ambiguity exceeds shallow Indic scripts’. We measure only the English side (CMUdict homograph entropy 0.070 bits/type); we have no Indic pronunciation lexicon, so the cross-lingual direction is literature-consistent but *not* established by our instrument, and we report it as future work, not a result. (2) The matched code is a small-data demonstration ($\sim 1k$ sentences, vocabulary 2–3k); it understates removability, making $\rho = 0.64$ a lower bound. (3) The information floor is an LZMA estimate, an upper bound on intrinsic content. (4) This is not a quality claim; we measure compute and memory and do not contradict Schmidt et al. (2024). (5) Scope: eight languages, one parallel benchmark, text only; speech/multimodal cost is argued, not measured. (6) We prove no new theorem: the floor and its split are cited, not claimed.

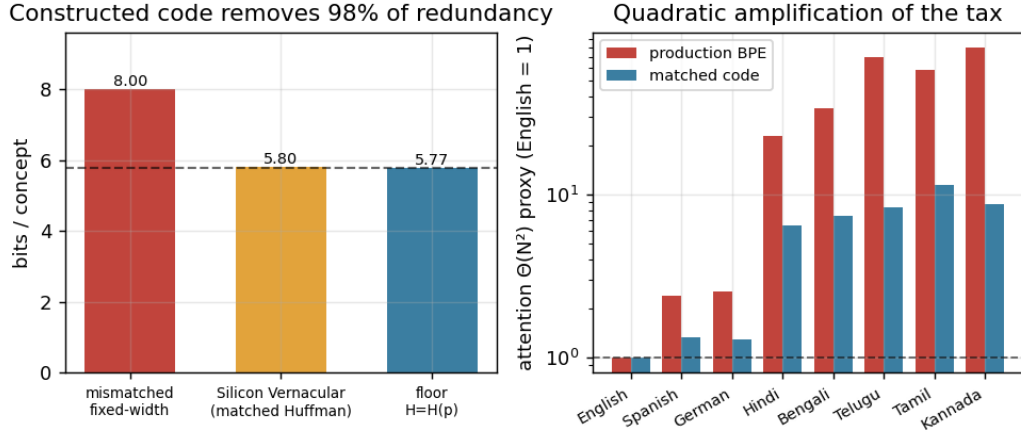


Figure 3: Left: a constructed matched code removes 98% of a controlled source’s redundancy, landing 0.036 bits above the entropy floor. Right: the $\Theta(N^2)$ attention-cost proxy (English = 1), production vs. matched—the token tax is quadratically amplified in compute.

8 Conclusion

The multilingual tokenization tax is, in the compute ledger, mostly *removable*: on real parallel text a script-matched code trained on a thousand sentences erases about two-thirds of it, and a script-fair floor shows the intrinsic content of the same sentences differs by under six percent. What remains—the organic grapheme-to-phoneme cost of a deep orthography—is real but lives on a different (multimodal) axis, and flips the ranking. We contribute the unifying ledger, the removable-versus-intrinsic attribution, a constructed code that reaches the entropy floor, and an open harness; we claim neither the floor, nor the optimal vocabulary, nor a model-quality benefit.

Reproducibility

All numbers and figures regenerate from a single command over public data (FLORES-200), with no GPU or model weights. Code, data pointers, and the pre-registration are provided as an anonymized supplement; the repository URL is withheld to preserve double-blind review.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. 2023. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Kaj Bostrom and Greg Durrett. 2020. Byte pair encoding is suboptimal for language model pretraining. In

Findings of the Association for Computational Linguistics: EMNLP 2020. ArXiv:2004.03720.

Thomas M. Cover and Joy A. Thomas. 2006. *Elements of Information Theory*, 2nd edition. Wiley-Interscience.

Mehmet Erdogan, Aditya Gorle, Shubham Chandak, Mert Pilanci, and Tsachy Weissman. 2026. An information-theoretic perspective on llm tokenizers. *arXiv preprint arXiv:2601.09039*.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. 2025. The foundations of tokenization: Statistical and computational concerns. In *International Conference on Learning Representations (ICLR)*. ArXiv:2407.11606.

IndicSuperTokenizer authors. 2025. IndicSuperTokenizer: Efficient tokenization for indic languages. *arXiv preprint arXiv:2511.03237*.

Tomasz Limisiewicz, Artidoro Pagnoni, Srinivasan Iyer, et al. 2026. Compute optimal tokenization. *arXiv preprint arXiv:2605.01188*.

Jessica Lundin et al. 2025. The token tax: Systematic bias in multilingual tokenization. *arXiv preprint arXiv:2509.05486*.

NLLB Team, Marta R. Costa-jussà, et al. 2022. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*. FLORES-200 evaluation benchmark.

Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, et al. 2024. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*.

Aleksandar Petrov, Emanuele La Malfa, Philip H.S. Torr, and Adel Bibi. 2023. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2305.15425.

- Nived Rajaraman, Jiantao Jiao, and Kannan Ramchandran. 2024. Toward a theory of tokenization in LLMs. *arXiv preprint arXiv:2404.08335*.
- Craig W. Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. 2024. Tokenization is more than compression. *arXiv preprint arXiv:2402.18376*.
- Claude E. Shannon. 1948. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423.
- Chaofan Tao, Qian Liu, Longxu Dou, Niklas Muenighoff, Zhongwei Wan, Ping Luo, Min Lin, and Ngai Wong. 2024. Scaling laws with vocabulary: Larger models deserve larger vocabularies. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2407.13623.
- Diego Torres and Richard Futrell. 2025. Clarifying orthography: Orthographic transparency as compressibility. *arXiv preprint arXiv:2505.13657*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. 2023. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. 2023. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2306.16842.